Análise teórica e metodologia experimental

Estruturas: Lista Simplesmente Encadeada, Árvore AVL e Lista Duplamente Encadeada. Esta versão substitui o Vetor Dinâmico. Os resultados antigos do vetor não são válidos para a lista dupla.

Passo 4 — Análise teórica (Big-O)

Hipóteses: as duas listas permanecem ordenadas por timestamp durante a inserção; a remoção recebe um timestamp e precisa procurá-lo; a listagem visita TODOS os n elementos. A AVL é uma árvore binária de busca balanceada.

Estrutura	Inserir	Remover	Listar ordenado
Lista simples	Melhor O(1) Pior O(n)	Melhor O(1) Pior O(n)	Melhor O(n) Pior O(n)
AVL	Melhor O(1) Pior O(log n)	Melhor O(1) Pior O(log n)	Melhor O(n) Pior O(n)
Lista dupla	Melhor O(1) Pior O(n)	Melhor O(1) Pior O(n)	Melhor O(n) Pior O(n)

Justificativa das listas: inserir/remover no início exige poucas operações; quando o timestamp está no fim, a busca pode percorrer n nós. Para listar todos os n registros, é obrigatório visitar cada nó, logo O(n) mesmo no melhor caso.

Justificativa da AVL: a altura é O(log n), portanto busca, inserção e remoção têm limite O(log n). Em casos favoráveis, uma operação pode ser concluída em O(1). A listagem em ordem percorre todos os nós e todas as ocorrências, totalizando O(n).

Observação: o ponteiro anterior da lista dupla facilita ajustar ligações ao remover um nó já localizado, mas NÃO elimina a busca O(n) por timestamp. Duplicatas são permitidas nas três estruturas.

Passo 5 — Dados de teste

São utilizados os arquivos timestamps_1000.txt, timestamps_5000.txt, timestamps_10000.txt, timestamps_50000.txt e timestamps_100000.txt, cada linha no formato YYYY-MM-DD HH:MM:SS. As três estruturas recebem exatamente os mesmos registros na ordem original, não pré-ordenada.

Passo 6 — Medição de tempo

O benchmark usa clock() da biblioteca <time.h> e calcula: ciclos estimados = ((clock_f − clock_i) / CLOCKS_PER_SEC) × frequência_base_Hz. Não se trata de contagem real de ciclos de hardware.

Processador utilizado: [PREENCHER MODELO EXATO]. Frequência base nominal: [PREENCHER EM Hz]. Compilação: gcc -std=c11 -Wall -Wextra -O0. Sistema operacional e versão do compilador: [PREENCHER].

Para remoção, primeiro inserimos n registros fora do trecho cronometrado e removemos os primeiros 10% dos timestamps na ordem da entrada. Para listagem, preenchemos a estrutura fora da medição e visitamos todos os n registros com callback sem impressão. Para inserção, cronometramos a inserção dos n registros.

Passo 7 — Dez repetições e mediana

Para cada estrutura × tamanho × operação são executadas 10 repetições independentes. São 3 × 5 × 3 × 10 = 450 medições brutas. Ordenam-se os 10 valores; mediana = (5º + 6º) / 2. Também são informados mínimo e máximo.

Os resultados serão gerados após executar o novo benchmark. Os CSVs e gráficos antigos, que continham o Vetor Dinâmico, foram preservados APENAS como histórico e não devem ser usados na conclusão atualizada.

Gráficos e análise após a execução

O analisar.py produz: (1) Inserção; (2) Remoção; (3) Listagem ordenada; (4) Custo ponderado = 0,70 × inserção + 0,15 × remoção + 0,15 × listagem. Também calcula ajustes de potência y = a × n^b em escala log-log e R^2 quando todas as medianas são positivas.

ATENÇÃO: clock() pode retornar duração zero em operações muito rápidas; isso impossibilita o ajuste logarítmico dessas séries. Não substituir zeros por números inventados. Relatar a limitação e, se necessário, revisar o protocolo de medição com o grupo/professor.

Situação da entrega

Código atualizado e compilado; teste de carregamento e listagem executado com 1.000 registros. A coleta experimental completa, os gráficos novos e a conclusão baseada em medições da lista dupla dependem da execução do benchmark no computador escolhido pelo grupo.